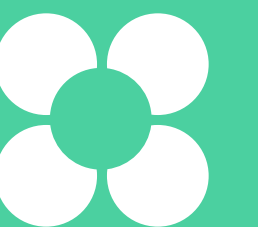
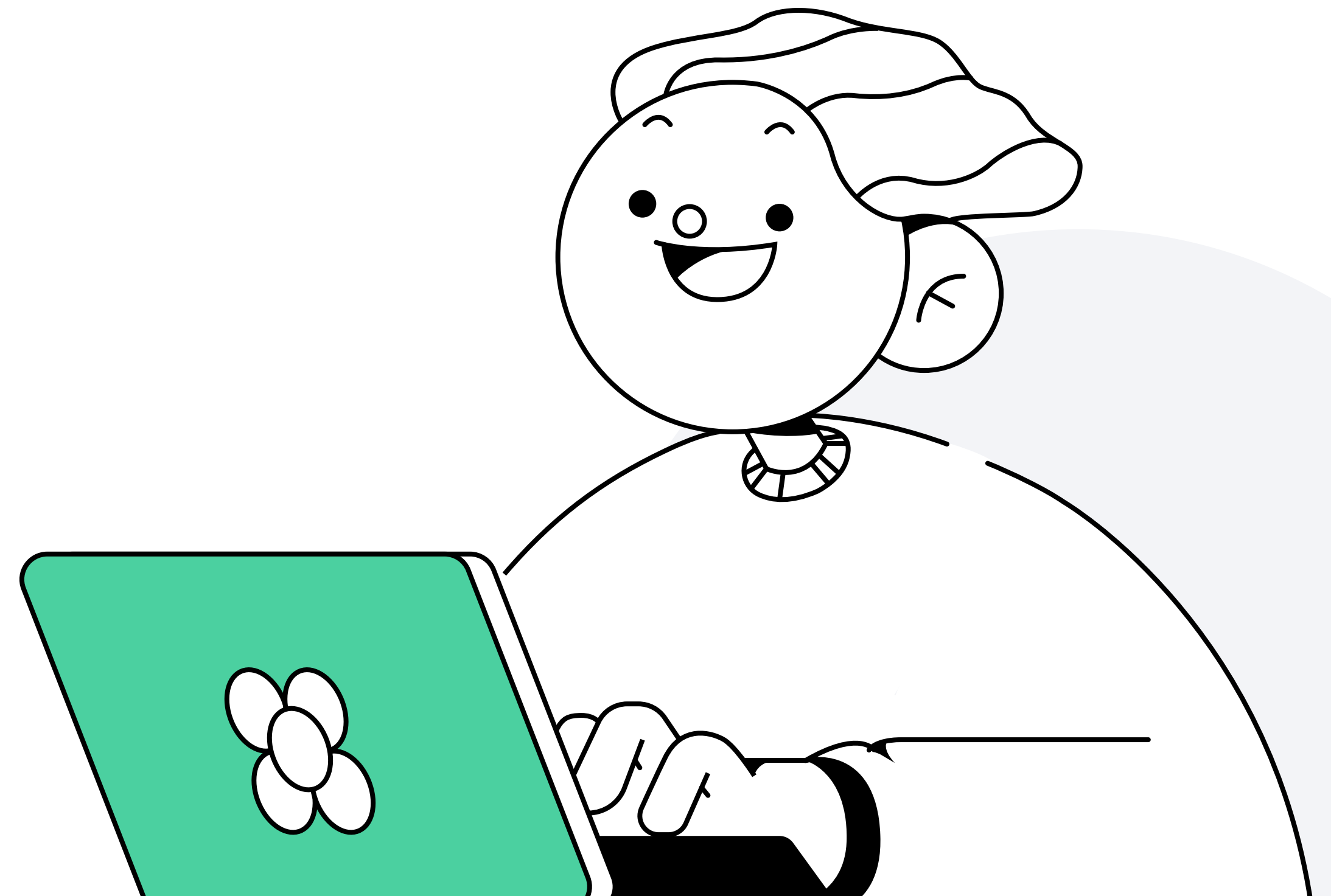


Одномерные массивы

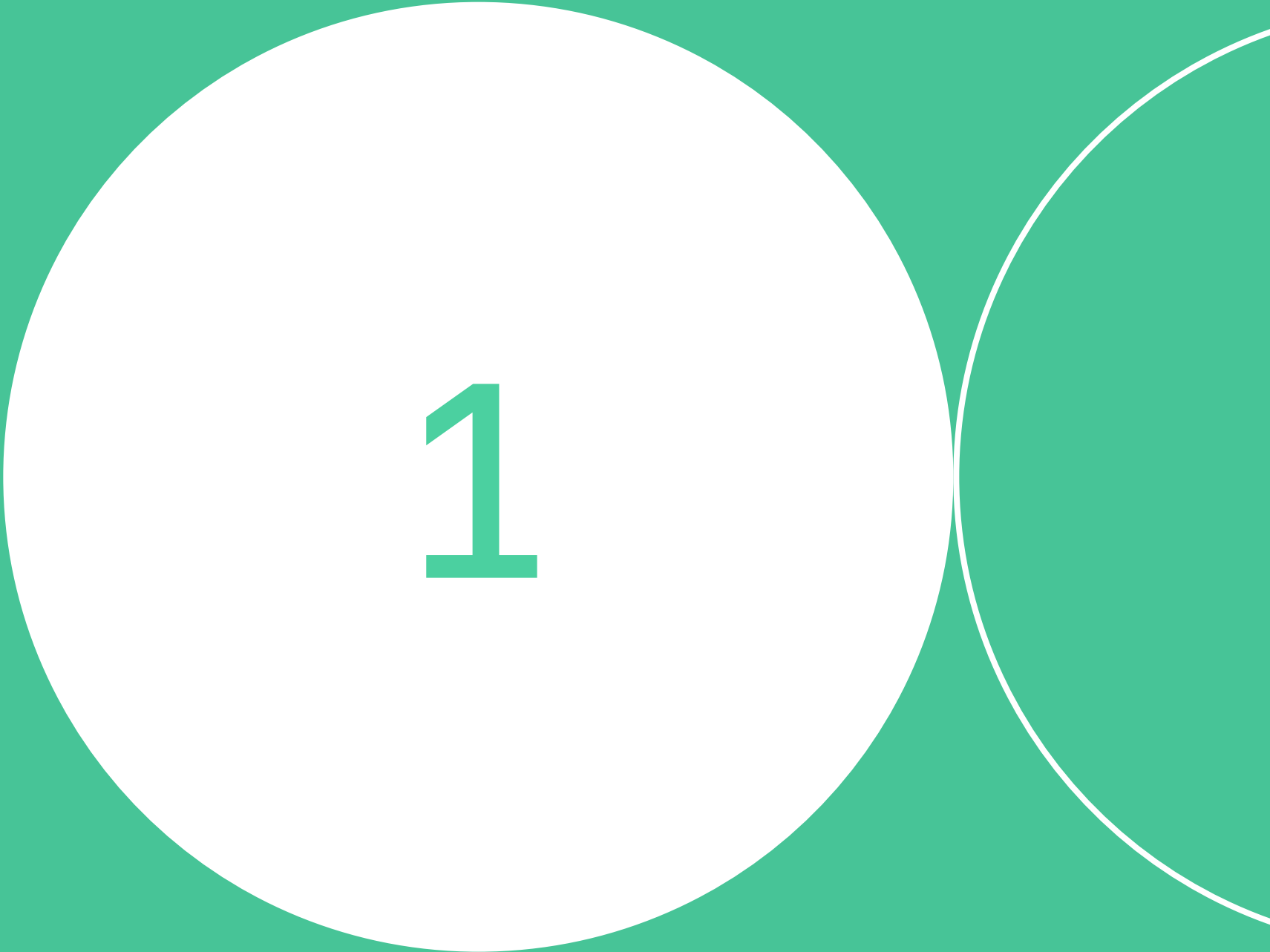


План занятия

- 1 Основы работы с массивами
- 2 Работа с массивами в цикле. Цикл foreach



Основы работы с массивами

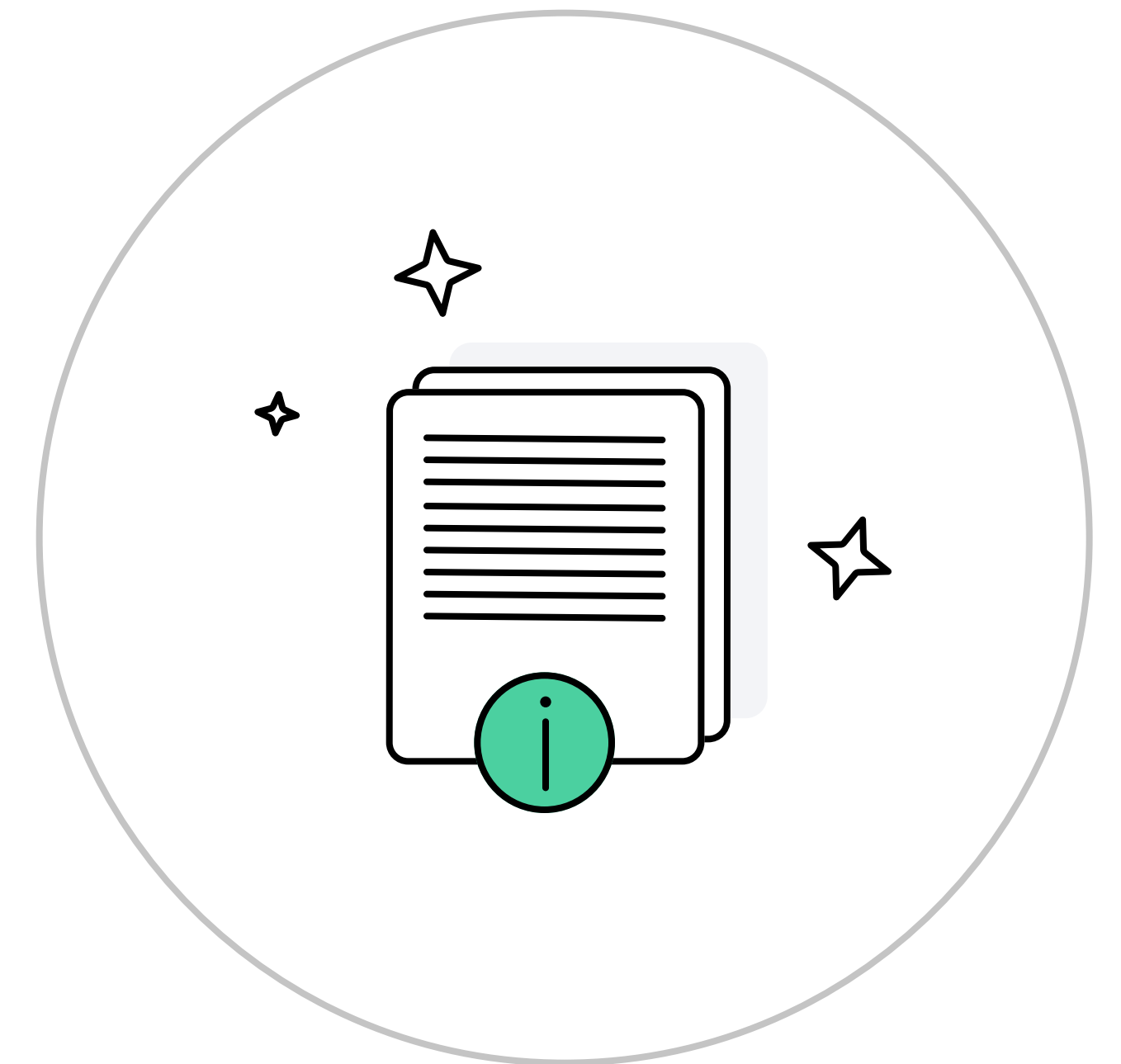


1

Что такое массив. Одномерные массивы

- **Массив** — это набор элементов (переменных) одного типа
- **Индекс** — это позиция элемента в массиве. Начинается индекс с нуля

В одномерных массивах для определения местоположения элемента нужно указать значение одного индекса



Структура массива

Для объявления массива надо указать тип, размер и имя. Массив можно создать двумя способами:

```
тип имя_переменной[] = new тип[размер];
```

ИЛИ

```
тип[] имя_переменной = new тип[размер];
```

- **тип** — базовый тип элементов массива
- **размер** — число элементов массива (выделение памяти для элементов).
Элементы массива могут быть любого типа, допустимого в языке Java

Структура массива

После определения размера выделяется память для массива. Выделенная память не может быть пустой: для разных типов данных память заполняется разными значениями.

При выделении памяти элементы массива заполняются:

- для числовых типов — нулевыми значениями
- для логического типа **boolean** — значениями **false**
- для ссылочных типов — значениями **null**

Инициализация массива

Предположим, у нас есть список цен на товары.

Нам нужно список сохранить. Для этого определим массив типа `int` и дадим размер массиву (например, у нас список из 5 цен):

```
int[] arr = new int [5];
```

У нас есть массив с определённым размером и нулевыми значениями, осталось заполнить его значениями.

Для этого нужно взять ячейки массива и присвоить им значения. Заполним первую ячейку:

```
arr[0] = 5;
```

Инициализация массива

Заполним остальные ячейки:

```
arr[1] = 3;  
arr[2] = 15;  
arr[3] = 100;  
arr[4] = 1;
```

Также можно заполнить ячейки значениями вместе с инициализацией массива:

```
int arr2[] = {5, 3, 15, 100, 1};
```


Работа с массивами

Давайте выведем полностью весь массив на экран, используя метод **toString**, который находится в классе **Arrays** — это утильный класс методов в Java для работы с массивами.

Метод **toString** в массивах наследуется от класса **Object** и не несёт никакой пользы, кроме адреса в памяти:

```
int[] arr = new int [5];  
arr[0] = 5;  
arr[1] = 3;  
arr[2] = 15;  
arr[3] = 100;  
arr[4] = 1;  
System.out.println(Arrays.toString(arr)); // выведет в консоль [5, 3, 15, 100, 1]
```

Сравнение массивов

Нельзя просто сравнить два массива на предмет равенства, потому что в этом случае Java сравнивает адреса, как в ситуации с ссылочным типом:

```
System.out.println(arr == arr2); // выведет в консоль false
```

Второй знакомый нам вариант сравнения через **equals** также не сработает:

```
System.out.println(arr.equals(arr2)); // выведет в консоль false
```

Сравнение массивов

Правильное сравнение массивов достигается через вспомогательные методы класса **Arrays**:

```
System.out.println(Arrays.equals(arr, arr2)); // в результате true
```


Инициализация одномерного массива классов

В качестве типа ячейки для массива можно использовать любой другой тип:

```
String[] arr = {"Petya", "Olya", "Tanya"};
```

В качестве типа ячейки можно использовать любой нами созданный тип. Имея созданный тип данных `Singer`, создадим массив данных двумя способами:

```
Singer[] arr2 = new Singer[4]
```

```
Singer[] arr2 = {  
    new Singer("Petya", 8);  
    new Singer("Olya", 15, 4);  
}
```

Работа с массивами в цикле. Цикл `foreach`



2

Работа с массивами в цикле

Посмотрим, как массив помогает шаблонно обрабатывать целые блоки данных.

Наша задача — вывести на экран все имена, которые начинаются не с буквы Р

```
String[] names = {  
    "Petya",  
    "Olya",  
    "Anya",  
    "Pavel",  
    "Oleg"  
};
```


Работа с массивами с помощью цикла for

Используем для выполнения задачи цикл **for**.

Он будет повторять нашу команду для разных значений переменной **определённое** количество раз в зависимости от количества переменных.

Напишем программу, где:

- **i** — имя счётчика
- **.length** — запрос размера массива
- **startsWith()** — метод, который проверяет, не начинается ли этот текст с какого-то конкретного текста
- **!** — логическое отрицание

```
String[] names = {  
    "Petya",  
    "Olya",  
    "Anya",  
    "Pavel",  
    "Oleg"  
};  
  
for (int i = 0; i < names.length; i++) {  
    if (!names[i].startsWith("P")) {  
        System.out.println(names[i]);  
    }  
}
```

Работа с массивами с помощью цикла `foreach`

Для выполнения этой же задачи мы можем использовать цикл **`foreach`**.

В случае с **`foreach`** мы не берём в расчёт, что есть какое-то определённое количество ячеек. Java просматривает каждую ячейку и кладёт значение в переменную **`String name`**

```
String[] names = {  
    "Petya",  
    "Olya",  
    "Anya",  
    "Pavel",  
    "Oleg"  
};  
  
for (String name : names) {  
    if (!name.startsWith("P")) {  
        System.out.println(name);  
    }  
}
```


Класс **Arrays**

Как вы уже поняли, Java позаботилась о нас и создала целый класс **Arrays** для работы с массивами.

Ниже представлены некоторые методы этого класса, которые пригодятся вам в работе.

- **copyOf()** — копирует массив
- **copyOfRange()** — копирует часть массива
- **toString()** — возвращает все элементы в виде одной строки
- **sort()** — сортирует массив
- **binarySearch()** — ищет элемент методом бинарного поиска
- **fill()** — заполняет массив переданным значением
- **equals()** — проверяет на идентичность массивы
- **asList()** — возвращает массив как коллекцию